Remote Debugging a State-Driven Flink PTF UDF

Flink TaskManagers are JVM processes, so you can enable **Java remote debugging (JDWP)** and attach your IDE (VS Code or IntelliJ IDEA) to them.

Table of Contents

- **1.0 Quick Start**
- **2.0 How It Works**
  - **2.1 JDWP on the Flink TaskManager**
  - **2.2 Supplemental RBAC**
  - **2.3 Port Forwarding (automated)**
  - **2.4 IDE Debug Configurations**
    - **2.4.1 VS Code (`.vscode/launch.json`)**
    - **2.4.2 IntelliJ IDEA (`.idea/runConfigurations/`)**
- **3.0 Important Caveats**
- **4.0 For Confluent Cloud**

> JDWP is already enabled on the TaskManager, debug configurations for both VS Code and IntelliJ IDEA are in place, and port-forwarding is automated as a pre-launch task. Once your cluster is running and your UDF is deployed, use the **"Attach to Flink TaskManager"** debug configuration to attach to the remote JVM and hit your breakpoints.
>
> **Note:** Make sure you select the **"Attach to Flink TaskManager"** configuration before launching the debugger. Starting a *launch* (rather than *attach*) configuration will attempt to run a local Java process, which fails because UDFs have no `main()` method.
>
> - **VS Code:** Open the **Run and Debug** panel (⇧⌘D), select **"Attach to Flink TaskManager"** from the dropdown, then press **F5**.
> - **IntelliJ IDEA:** Open the **Run/Debug Configurations** dropdown (top-right toolbar), select **"Attach to Flink TaskManager"**, then click **Debug** (^D / Shift+F9).

1.0 Quick Start

1. **Deploy** the full stack and your UDF:

   ```
   make cp-up                # Confluent Platform + Kafka UI
   make flink-up             # Flink Operator + CMF + Flink session
   cluster
   make deploy-cp-ptf-udf-state-driven   # Build UDF JAR, copy to Flink
   pods, submit SQL
   ```

2. **Set a breakpoint** in your UDF — open `UserEventEnricher.java` and click in the gutter at the first line of the `eval()` method:

```
String eventType = input.getFieldAs("event_type");
```

3. Select the **"Attach to Flink TaskManager"** debug configuration in your IDE:
   - **VS Code:** Open **Run and Debug** (⇧⌘D) and choose it from the dropdown
   - **IntelliJ IDEA:** Choose it from the **Run/Debug Configurations** dropdown (top-right toolbar)
4. **Start the debugger** — your IDE will port-forward to the TaskManager pod automatically, then attach:
   - **VS Code:** Press **F5**
   - **IntelliJ IDEA:** Click **Debug** (^D / Shift+F9)
5. **Send a test event** to trigger the breakpoint:

```
make produce-user-events-record
```

6. **Debug** — your IDE pauses at your breakpoint. Inspect `input`, `state`, and local variables, step through the session logic, and watch `state.sessionId` and `state.eventCount` update as you step over lines.

2.0 How It Works

2.1 JDWP on the Flink TaskManager

> JDWP (Java Debug Wire Protocol) is the protocol used for communication between a debugger (like VS Code or IntelliJ) and a Java Virtual Machine (JVM) being debugged. It's part of the Java Platform Debugger Architecture (JPDA).
>
> Key points:
>
> - **Purpose:** Defines the format of requests and replies between the debugger front-end and the JVM.
> - **Transport:** Typically runs over a socket connection (TCP/IP), which is what enables **remote debugging** — the debugger and JVM can be on different machines.
> - **How it's enabled:** You pass JVM arguments like:
>
> `-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005`
>
> - `transport=dt_socket` — use TCP sockets
> - `server=y` — the JVM listens for a debugger to attach
> - `suspend=n` — don't pause the JVM on startup waiting for a debugger (`y` would pause)
> - `address=*:5005` — listen on port `5005`

The FlinkDeployment CR (`k8s/base/flink-basic-deployment.yaml`) includes the JDWP agent flag in the Flink configuration:

```
flinkConfiguration:
  env.java.opts.taskmanager: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005"
  heartbeat.timeout: "300000"
```

- `suspend=n` — the JVM starts normally (use `suspend=y` if you need it to wait for the debugger before processing)
- `address=*:5005` — listens on port `5005`
- `heartbeat.timeout` — increased to `5 minutes` so pausing at breakpoints doesn't kill the TaskManager

2.2 Supplemental RBAC

Flink's Kubernetes session-cluster executor needs to GET the JobManager REST service (e.g. `flink-basic-rest`) to submit jobs. The Helm-managed Role for the `flink` ServiceAccount grants access to pods, configmaps, and deployments — but **not services**.

The manifest `k8s/base/flink-rbac.yaml` fills this gap with a supplemental Role and RoleBinding:

```
rules:
  - apiGroups: [""]
    resources: ["services"]
    verbs: ["get", "list", "watch"]
```

This is additive — Kubernetes merges permissions across all Roles bound to the same subject, so the Helm-managed Role is unaffected. The `flink-rbac` Makefile target applies this manifest and runs automatically as a dependency of `flink-deploy`.

2.3 Port Forwarding (automated)

Both IDEs run a helper script (`scripts/port-forward-taskmanager.sh`) as a pre-launch / "Before launch" step that:

1. Kills any existing port-forward on port `5005`
2. Discovers the TaskManager pod by label (`component=taskmanager`)
3. Starts `kubectl port-forward` in the background (detached via `nohup`/`disown` so it survives after the task shell exits)
4. Waits for port `5005` to be listening
5. Exits with success — the IDE then proceeds to attach the debugger

VS Code triggers the script via a task in `.vscode/tasks.json`:

```
{
    "label": "Port Forward Flink TaskManager",
    "type": "shell",
    "command": "${workspaceFolder}/scripts/port-forward-taskmanager.sh"
}
```

IntelliJ IDEA triggers the same script via a Shell Script run configuration (`.idea/runConfigurations/Port_Forward_Flink_TaskManager.xml`) that is wired as a "Before launch" step in the debug configuration.

If you need to port-forward manually (e.g., outside your IDE):

```
kubectl port-forward -n confluent <taskmanager-pod> 5005:5005
```

2.4 IDE Debug Configurations

Both IDEs ship pre-configured **"Attach to Flink TaskManager"** and **"Debug UDF Tests"** configurations. Each attach configuration wires the port-forward script as a pre-launch step so the entire flow is one click.

2.4.1 VS Code (`.vscode/launch.json`)

```
{
  "type": "java",
  "name": "Attach to Flink TaskManager",
  "request": "attach",
  "hostName": "localhost",
  "port": 5005,
  "projectName": "app",
  "preLaunchTask": "Port Forward Flink TaskManager"
}
```

The `projectName` is `"app"` because that is the Gradle subproject name defined in `examples/ptf_udf_state_driven/java/settings.gradle.kts`. This tells the debugger which classpath and source roots to use for resolving breakpoints.

2.4.2 IntelliJ IDEA (`.idea/runConfigurations/`)

IntelliJ run configurations are stored as XML and are automatically recognized when you open the project:

Run Configuration	What it does
Port Forward Flink TaskManager	Shell Script config that runs `scripts/port-forward-taskmanager.sh` to `kubectl port-forward` port `5005` to the TaskManager pod
Attach to Flink TaskManager	Remote JVM Debug config that attaches to `localhost:5005`. The port-forward config runs automatically as a "Before launch" task
Debug UDF Tests	Gradle `test` task scoped to `examples/ptf_udf_state_driven/java`

> **Note:** The Shell Script run configuration requires the **Shell Script** plugin, which is bundled with IntelliJ IDEA 2020.2+.

3.0 Important Caveats

- **TaskManager must be running** — the TaskManager pod only exists while a Flink job is active. Deploy your UDF first (`make deploy-cp-ptf-udf-state-driven`) before attaching the debugger
- **Source must match** — the local code you have open in your IDE must match the JAR deployed to the cluster, or breakpoints won't align
- **Timeouts** — pausing too long at a breakpoint can trigger Flink's heartbeat timeout, causing the TaskManager to be considered dead. The `heartbeat.timeout` is already set to `5 minutes` in the Flink config
- **Single TaskManager** — if you have multiple TaskManagers, you're only attached to one. Debug with `parallelism=1` to keep things simple
- **`suspend=y`** is useful if you need to debug the `open()` lifecycle method, since it pauses the JVM before any processing starts

4.0 For Confluent Cloud

*Remote debugging is **NOT POSSIBLE** on Confluent Cloud — you don't have access to the underlying JVMs. For that environment, stick with local MiniCluster integration tests or logging.*